

COMP90038
Algorithms and Complexity

Cezary Kaliszyk
(thanks to Harald Søndergaard)

Lecture 19

Dynamic Programming: Warshall and Floyd

Dynamic Programming and Graphs

Last lecture

Dynamic programming

Today: applied to two graph problems

- Computing the transitive closure of a directed graph
- Finding shortest distances in weighted directed graphs

Warshall's Algorithm

Computes the transitive closure

- of a binary relation (or a directed graph)
- presented as a matrix

Warshall's Algorithm

Computes the transitive closure

- of a binary relation (or a directed graph)
- presented as a matrix

Definition

An edge (a, z) is in the transitive closure of graph G iff there is a path in G from a to z.

Warshall's Algorithm

Computes the transitive closure

- of a binary relation (or a directed graph)
- presented as a matrix

Definition

An edge (a, z) is in the transitive closure of graph G iff there is a path in G from a to z.

Originally not dynamic programming

But easy to understand as DP

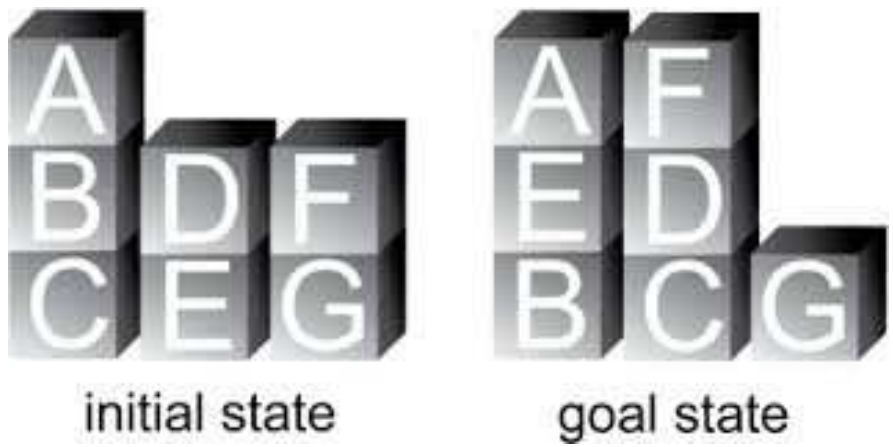
Transitive Closure over a State Space

Useful for analysing reachability

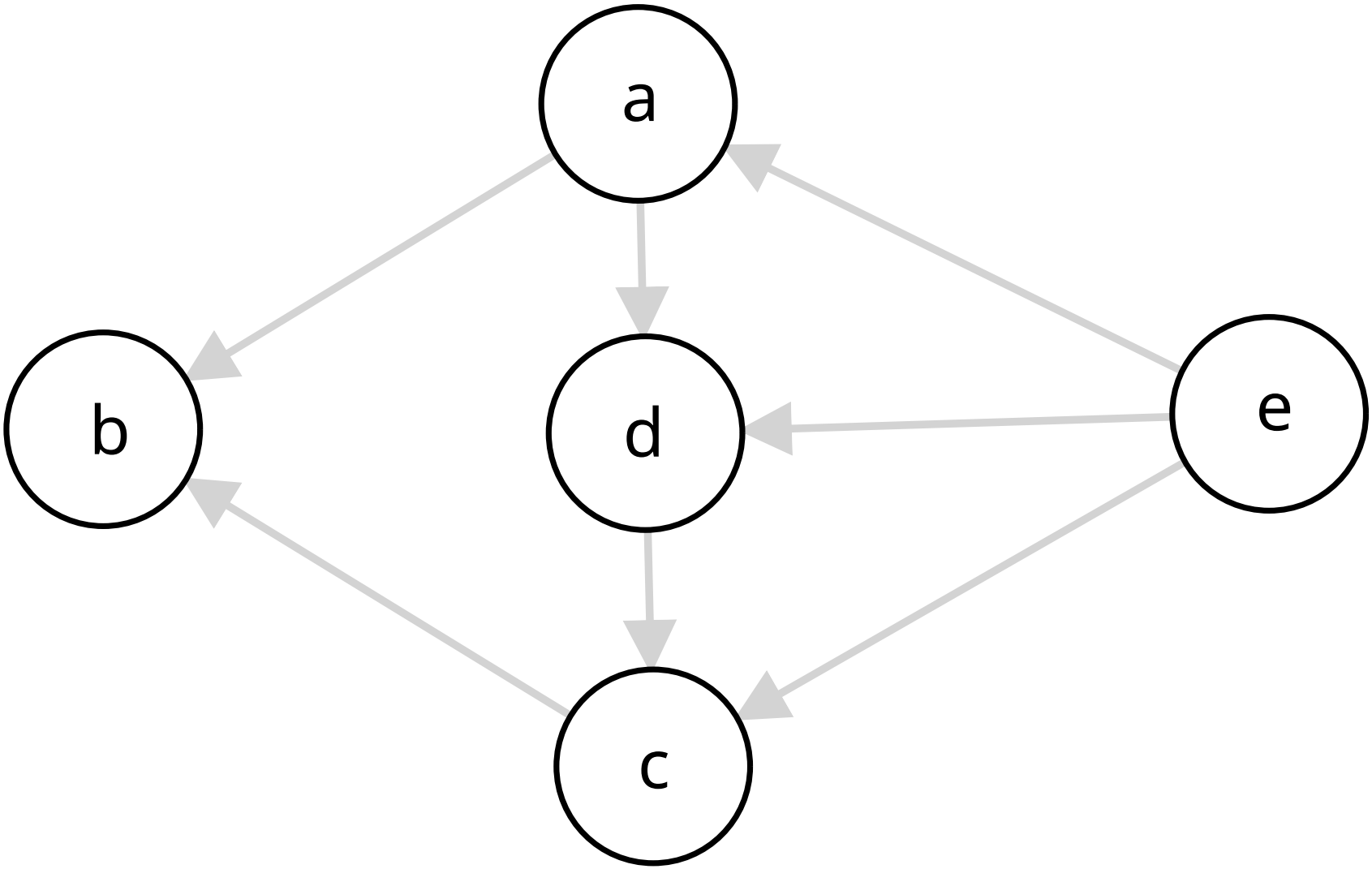
applications where we want to see if some “state” (or node) is reachable from some “initial state/node”

Example

Is there a sequence of steps that will allow the containers of a ship to be reorganised in a certain way?



Transitive Closure Example



Reasoning about Transitive Closure

Assume the nodes of graph G are numbered from 1 to n.

Ask the question: Is there a path from node i to node j using only nodes that are no larger than some k as "stepping stones"?

Reasoning about Transitive Closure

Assume the nodes of graph G are numbered from 1 to n.

Ask the question: Is there a path from node i to node j using only nodes that are no larger than some k as "stepping stones"?

Such a path either uses node k as a stepping stone, or it doesn't.

So an answer is: There is such a path if and only if we can

step from i to j using only nodes $\leq k-1$, or

step from i to k using only nodes $\leq k-1$, and then
step from k to j using only nodes $\leq k-1$.

Warshall's Algorithm

If G's adjacency matrix is A then we can express the recurrence relation as

$$\begin{aligned} R^0_{ij} &= A[i,j] \\ R^k_{ij} &= R^{k-1}_{ij} \text{ or } (R^{k-1}_{ik} \text{ and } R^{k-1}_{kj}) \end{aligned}$$

Warshall's Algorithm

If G's adjacency matrix is A then we can express the recurrence relation as

$$\begin{aligned} R^0_{ij} &= A[i,j] \\ R^k_{ij} &= R^{k-1}_{ij} \text{ or } (R^{k-1}_{ik} \text{ and } R^{k-1}_{kj}) \end{aligned}$$

This gives us a dynamic-programming algorithm:

```
function WARSHALL(A[·,·], n)
    R^0 ← A
    for k ← 1 to n do
        for i ← 1 to n do
            for j ← 1 to n do
                R^k[i,j] ← R^{k-1}[i,j] or (R^{k-1}[i,k] and R^{k-1}[k,j])
    return R^n
```

Warshall's Algorithm

If we allow input A to be used for the output, we can simplify things.

Namely, if $R^{k-1}[i,k]$ (that is, $A[i,k]$) is 0 then the assignment is doing nothing. And if it is 1, and if $A[k,j]$ is also 1, then $A[i,j]$ gets set to 1.

Warshall's Algorithm

If we allow input A to be used for the output, we can simplify things.

Namely, if $R^{k-1}[i,k]$ (that is, $A[i,k]$) is 0 then the assignment is doing nothing. And if it is 1, and if $A[k,j]$ is also 1, then $A[i,j]$ gets set to 1. So:

```
for k ← 1 to n do
    for i ← 1 to n do
        for j ← 1 to n do
            if A[i,k] then
                if A[k,j] then
                    A[i,j] ← 1
```

But now we notice that $A[i,k]$ does not depend on j, so testing it can be moved outside the innermost loop.

Warshall's Algorithm

This leads to a better version of Warshall's algorithm:

```
for k ← 1 to n do
    for i ← 1 to n do
        if A[i, k] then
            for j ← 1 to n do
                if A[k, j] then
                    A[i, j] ← 1
```

If each row in the matrix is represented as a bit-string, then the innermost for loop (and j) can be gotten rid of—instead of iterating, just apply the "bitwise or" of row k to row i.

Example of Running Warshall's Algorithm (1)

Example of Running Warshall's Algorithm (1)

$$\begin{pmatrix} - & 1 & - & 1 & - \\ - & - & - & - & - \\ - & 1 & - & - & - \\ - & - & 1 & - & - \\ 1 & - & 1 & 1 & - \end{pmatrix} \xrightarrow{Preprocessing} R_0 = \left(\begin{array}{c|cccc} 0 & 1 & 0 & 1 & 0 \\ \hline 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \end{array}\right) \rightarrow R_1 = \left(\begin{array}{c|c|ccc} 0 & 1 & 0 & 1 & 0 \\ \hline 0 & 0 & 0 & 0 & 0 \\ \hline 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 \end{array}\right) \rightarrow R_2 = \left(\begin{array}{cc|c|cc} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ \hline 0 & 1 & 0 & 0 & 0 \\ \hline 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 \end{array}\right)$$

$$\rightarrow R_3 = \left(\begin{array}{ccc|c|c} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ \hline 0 & 1 & 1 & 0 & 0 \\ \hline 1 & 1 & 1 & 1 & 0 \end{array}\right) \rightarrow R_4 = \left(\begin{array}{cccc|c} 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ \hline 1 & 1 & 1 & 1 & 0 \end{array}\right) \rightarrow R_5 = \begin{pmatrix} 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 \end{pmatrix} \xrightarrow{Solution} \begin{pmatrix} 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 \end{pmatrix}$$

Example of Running Warshall's Algorithm (2)

	1	2	3	4	5	6	7
1		1			1		
2			1				
3						1	1
4			1				
5		1				1	
6				1			1
7				1			

Example of Running Warshall's Algorithm (2)

	1	2	3	4	5	6	7
1		1	1		1		
2			1				
3						1	1
4			1				
5		1	1			1	
6				1			1
7				1			

Example of Running Warshall's Algorithm (2)

	1	2	3	4	5	6	7
1		1	1		1	1	1
2			1			1	1
3						1	1
4			1			1	1
5		1	1			1	1
6				1			1
7				1			

Example of Running Warshall's Algorithm (2)

	1	2	3	4	5	6	7
1		1	1		1	1	1
2			1			1	1
3						1	1
4			1			1	1
5		1	1			1	1
6			1	1		1	1
7			1	1		1	1

Example of Running Warshall's Algorithm (2)

	1	2	3	4	5	6	7
1		1	1		1	1	1
2			1			1	1
3						1	1
4			1			1	1
5		1	1			1	1
6			1	1		1	1
7			1	1		1	1

Example of Running Warshall's Algorithm (2)

	1	2	3	4	5	6	7
1		1	1	1	1	1	1
2			1	1		1	1
3			1	1		1	1
4			1	1		1	1
5		1	1	1		1	1
6			1	1		1	1
7			1	1		1	1

Example of Running Warshall's Algorithm (2)

	1	2	3	4	5	6	7
1		1	1	1	1	1	1
2			1	1		1	1
3			1	1		1	1
4			1	1		1	1
5		1	1	1		1	1
6			1	1		1	1
7			1	1		1	1

Analysis of Warshall's Algorithm

Straightforward Analysis

$\Theta(n^3)$; no difference between the best, average, and worst cases

Good for dense graphs (tight inner loop)

- However, it is not the best transitive-closure algorithm to use for sparse graphs.

Analysis of Warshall's Algorithm

Straightforward Analysis

$\Theta(n^3)$; no difference between the best, average, and worst cases

Good for dense graphs (tight inner loop)

- However, it is not the best transitive-closure algorithm to use for sparse graphs.
- DFS from each node v in turn, keeping track of which nodes are reached from v.

Floyd's Algorithm:

Problem: All-Pairs Shortest-Paths

For weighted graphs with positive weights

Floyd's Algorithm:

Problem: All-Pairs Shortest-Paths

For weighted graphs with positive weights

Works for:

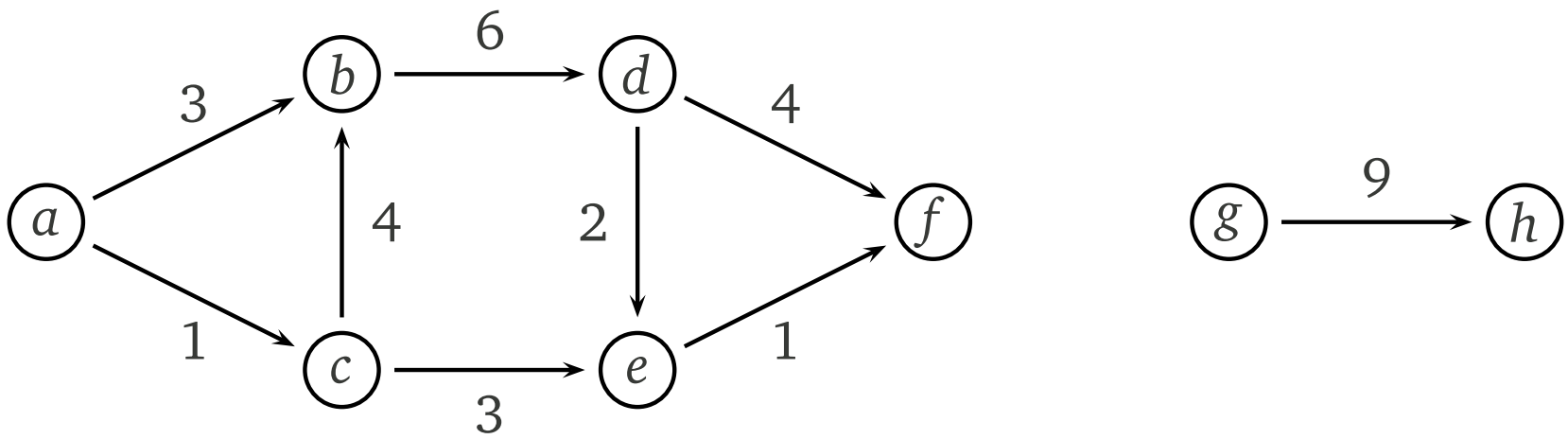
- directed and undirected graphs
- (sometimes works with non-positive weights, but not always)

Input:

- weight matrix W that holds all the edges' weights
- (for technical reasons, if there is no edge from node i to node j, we let $W[i, j] = \infty$)

We will construct the distance matrix D, step by step.

Floyd's Algorithm: All-Pairs Shortest-Paths



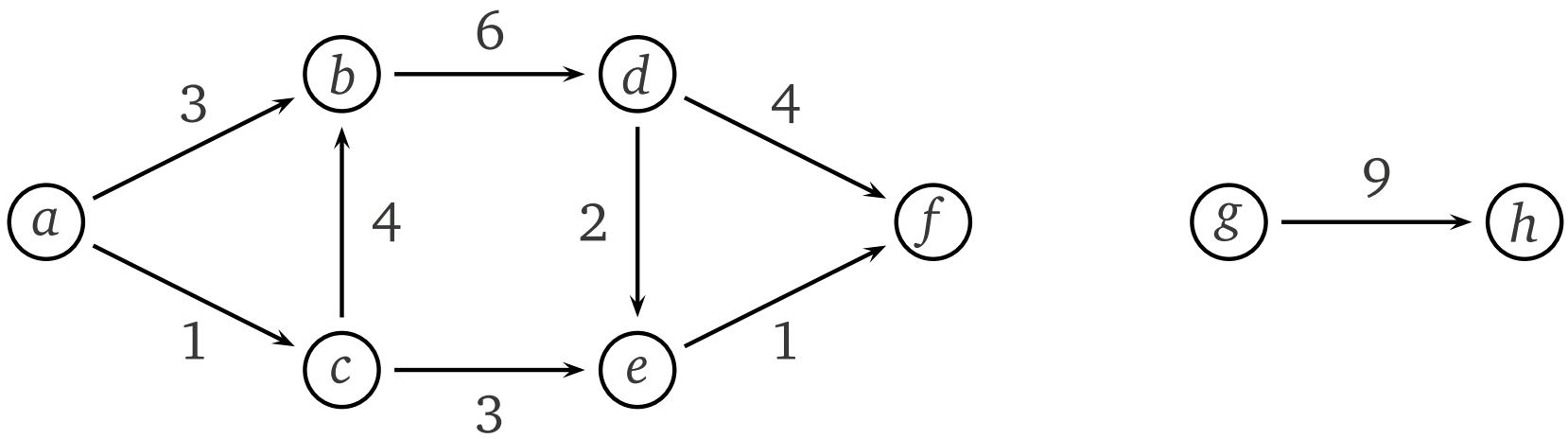
Task

construct a matrix D that gives the shortest path for each pair (u, v) of nodes

Examples

shortest path from a to f
shortest path from d to f
shortest path from f to h

Floyd's Algorithm: All-Pairs Shortest-Paths



Task

construct a matrix D that gives the shortest path for each pair (u, v) of nodes

Examples

shortest path from a to f has length 5
shortest path from d to f has length 3
shortest path from f to h is ∞

Floyd's Algorithm

Same problem decomposition as in Warshall

- What is the shortest path from node i to node j using only nodes $\leq k$ as "stepping stones"?
- (assume nodes are numbered 1 to n)

Floyd's Algorithm

Same problem decomposition as in Warshall

- What is the shortest path from node i to node j using only nodes $\leq k$ as "stepping stones"?
- (assume nodes are numbered 1 to n)

use node k as a stepping stone, or we avoid it

- step from i to j using only nodes $\leq k-1$, or
- step from i to k using only nodes $\leq k-1$, and then step from k to j using only nodes $\leq k-1$.

Floyd's Algorithm

If G's weight matrix is W then we can express the recurrence relation for minimal distances as follows:

$$\begin{aligned} D^0_{ij} &= W[i,j] \\ D^k_{ij} &= min(D^{k-1}_{ij}, D^{k-1}_{ik} + D^{k-1}_{kj}) \end{aligned}$$

Floyd's Algorithm

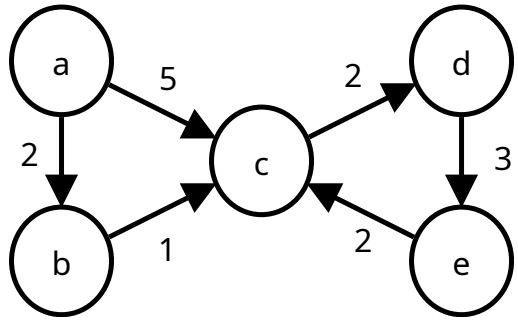
If G's weight matrix is W then we can express the recurrence relation for minimal distances as follows:

$$\begin{aligned} D^0_{ij} &= W[i,j] \\ D^k_{ij} &= min(D^{k-1}_{ij}, D^{k-1}_{ik} + D^{k-1}_{kj}) \end{aligned}$$

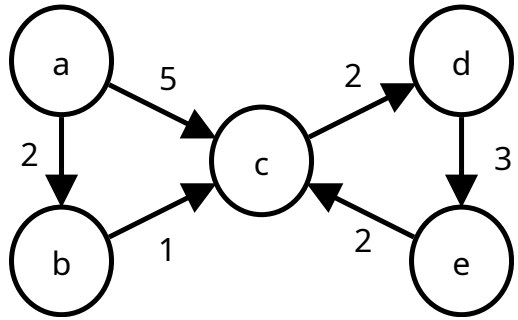
And then the algorithm follows easily:

```
function FLOYD(W[·,·], n)
    D ← W
    for k ← 1 to n do
        for i ← 1 to n do
            for j ← 1 to n do
                D[i,j] ← min(D[i,j], D[i,k] + D[k,j])
    return D
```

Example of Running Floyd's Algorithm (1)

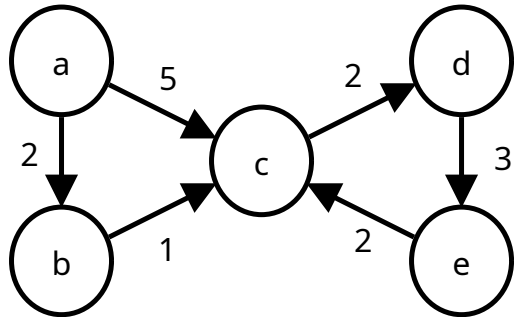


Example of Running Floyd's Algorithm (1)



$$A_0 = \begin{pmatrix} 0 & 2 & 5 & \infty & \infty \\ \infty & 0 & 1 & \infty & \infty \\ \infty & \infty & 0 & 2 & \infty \\ \infty & \infty & \infty & 0 & 3 \\ \infty & \infty & 2 & \infty & 0 \end{pmatrix}$$

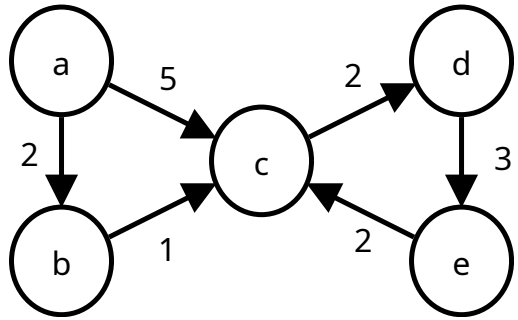
Example of Running Floyd's Algorithm (1)



$$A_0 = \begin{pmatrix} 0 & 2 & 5 & \infty & \infty \\ \infty & 0 & 1 & \infty & \infty \\ \infty & \infty & 0 & 2 & \infty \\ \infty & \infty & \infty & 0 & 3 \\ \infty & \infty & 2 & \infty & 0 \end{pmatrix}$$

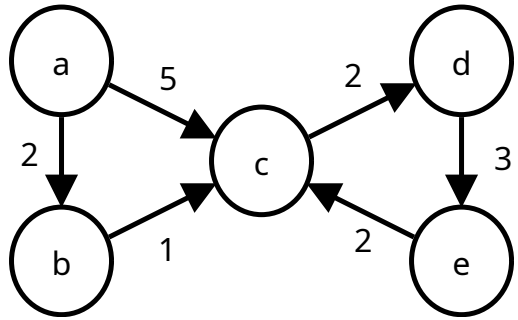
$$A_1 = A_0$$

Example of Running Floyd's Algorithm (1)



$$A_0 = \begin{pmatrix} 0 & 2 & 5 & \infty & \infty \\ \infty & 0 & 1 & \infty & \infty \\ \infty & \infty & 0 & 2 & \infty \\ \infty & \infty & \infty & 0 & 3 \\ \infty & \infty & 2 & \infty & 0 \end{pmatrix} \qquad A_1 = A_0 \qquad A_2 = \begin{pmatrix} 0 & 2 & 3 & \infty & \infty \\ \infty & 0 & 1 & \infty & \infty \\ \infty & \infty & 0 & 2 & \infty \\ \infty & \infty & \infty & 0 & 3 \\ \infty & \infty & 2 & \infty & 0 \end{pmatrix}$$

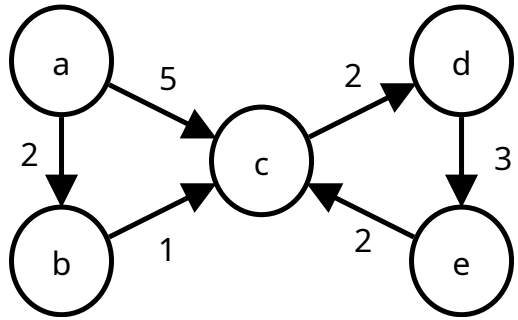
Example of Running Floyd's Algorithm (1)



$$A_0 = \begin{pmatrix} 0 & 2 & 5 & \infty & \infty \\ \infty & 0 & 1 & \infty & \infty \\ \infty & \infty & 0 & 2 & \infty \\ \infty & \infty & \infty & 0 & 3 \\ \infty & \infty & 2 & \infty & 0 \end{pmatrix} \qquad A_1 = A_0 \qquad A_2 = \begin{pmatrix} 0 & 2 & 3 & \infty & \infty \\ \infty & 0 & 1 & \infty & \infty \\ \infty & \infty & 0 & 2 & \infty \\ \infty & \infty & \infty & 0 & 3 \\ \infty & \infty & 2 & \infty & 0 \end{pmatrix}$$

$$A_3 = \begin{pmatrix} 0 & 2 & 3 & 5 & \infty \\ \infty & 0 & 1 & 3 & \infty \\ \infty & \infty & 0 & 2 & \infty \\ \infty & \infty & \infty & 0 & 3 \\ \infty & \infty & 2 & 4 & 0 \end{pmatrix}$$

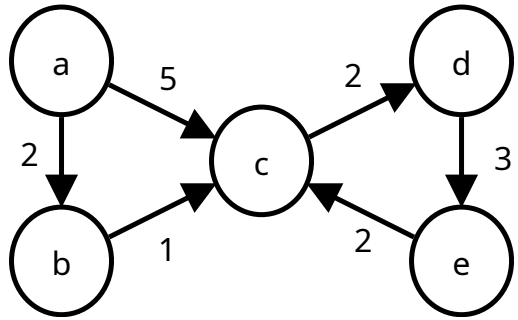
Example of Running Floyd's Algorithm (1)



$$A_0 = \begin{pmatrix} 0 & 2 & 5 & \infty & \infty \\ \infty & 0 & 1 & \infty & \infty \\ \infty & \infty & 0 & 2 & \infty \\ \infty & \infty & \infty & 0 & 3 \\ \infty & \infty & 2 & \infty & 0 \end{pmatrix} \qquad A_1 = A_0 \qquad A_2 = \begin{pmatrix} 0 & 2 & 3 & \infty & \infty \\ \infty & 0 & 1 & \infty & \infty \\ \infty & \infty & 0 & 2 & \infty \\ \infty & \infty & \infty & 0 & 3 \\ \infty & \infty & 2 & \infty & 0 \end{pmatrix}$$

$$A_3 = \begin{pmatrix} 0 & 2 & 3 & 5 & \infty \\ \infty & 0 & 1 & 3 & \infty \\ \infty & \infty & 0 & 2 & \infty \\ \infty & \infty & \infty & 0 & 3 \\ \infty & \infty & 2 & 4 & 0 \end{pmatrix} \qquad A_4 = \begin{pmatrix} 0 & 2 & 3 & 5 & 8 \\ \infty & 0 & 1 & 3 & 6 \\ \infty & \infty & 0 & 2 & 5 \\ \infty & \infty & \infty & 0 & 3 \\ \infty & \infty & 2 & 4 & 0 \end{pmatrix}$$

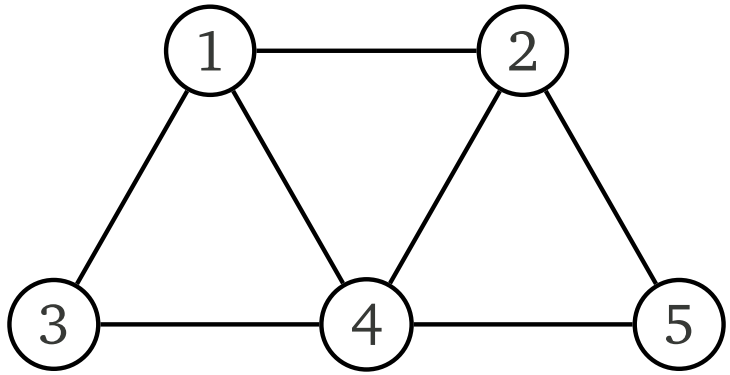
Example of Running Floyd's Algorithm (1)



$$A_0 = \begin{pmatrix} 0 & 2 & 5 & \infty & \infty \\ \infty & 0 & 1 & \infty & \infty \\ \infty & \infty & 0 & 2 & \infty \\ \infty & \infty & \infty & 0 & 3 \\ \infty & \infty & 2 & \infty & 0 \end{pmatrix} \qquad A_1 = A_0 \qquad A_2 = \begin{pmatrix} 0 & 2 & 3 & \infty & \infty \\ \infty & 0 & 1 & \infty & \infty \\ \infty & \infty & 0 & 2 & \infty \\ \infty & \infty & \infty & 0 & 3 \\ \infty & \infty & 2 & \infty & 0 \end{pmatrix}$$

$$A_3 = \begin{pmatrix} 0 & 2 & 3 & 5 & \infty \\ \infty & 0 & 1 & 3 & \infty \\ \infty & \infty & 0 & 2 & \infty \\ \infty & \infty & \infty & 0 & 3 \\ \infty & \infty & 2 & 4 & 0 \end{pmatrix} \quad A_4 = \begin{pmatrix} 0 & 2 & 3 & 5 & 8 \\ \infty & 0 & 1 & 3 & 6 \\ \infty & \infty & 0 & 2 & 5 \\ \infty & \infty & \infty & 0 & 3 \\ \infty & \infty & 2 & 4 & 0 \end{pmatrix} \quad A_5 = \begin{pmatrix} 0 & 2 & 3 & 5 & 8 \\ \infty & 0 & 1 & 3 & 6 \\ \infty & \infty & 0 & 2 & 5 \\ \infty & \infty & 5 & 0 & 3 \\ \infty & \infty & 2 & 4 & 0 \end{pmatrix}$$

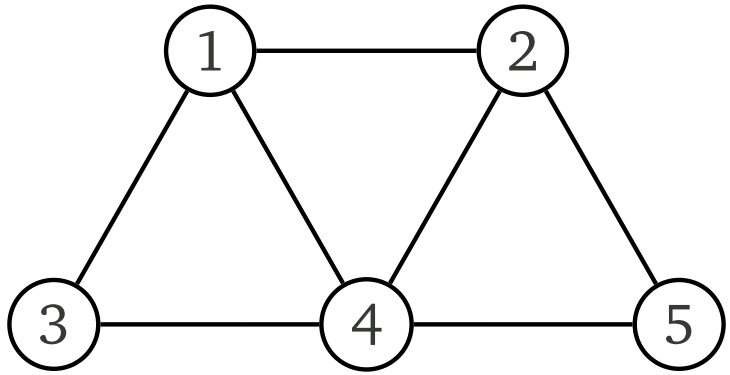
Example of Running Floyd's Algorithm (2)



The initial distance matrix (for the unweighted graph above)

	1	2	3	4	5
1	0	1	1	1	∞
2	1	0	∞	1	1
3	1	∞	0	1	∞
4	1	1	1	0	1
5	∞	1	∞	1	0

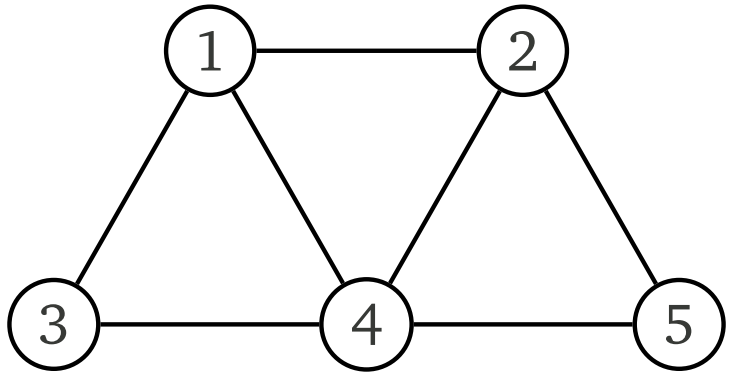
Example of Running Floyd's Algorithm (2)



Distance matrix after first round ($k = 1$)

	1	2	3	4	5
1	0	1	1	1	∞
2	1	0	2	1	1
3	1	2	0	1	∞
4	1	1	1	0	1
5	∞	1	∞	1	0

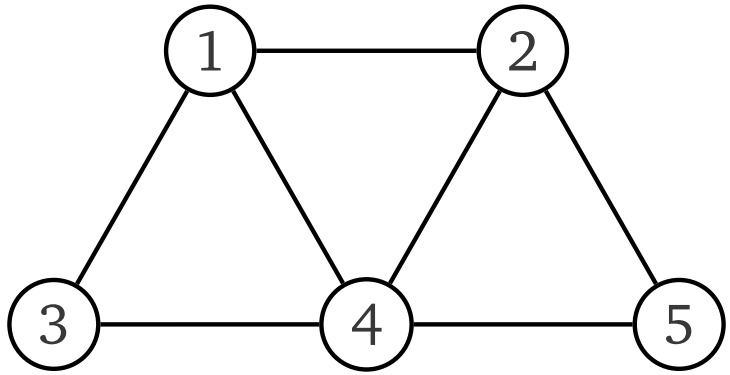
Example of Running Floyd's Algorithm (2)



Distance matrix after second round ($k = 2$).
In this example, no change happens in the following round ($k = 3$).

	1	2	3	4	5
1	0	1	1	1	2
2	1	0	2	1	1
3	1	2	0	1	3
4	1	1	1	0	1
5	2	1	3	1	0

Example of Running Floyd's Algorithm (2)



Distance matrix after fourth round ($k = 4$).
In this example, no further change happens for $k = 5$, so this is the final result.

	1	2	3	4	5
1	0	1	1	1	2
2	1	0	2	1	1
3	1	2	0	1	2
4	1	1	1	0	1
5	2	1	2	1	0

A Sub-Structure Property

Dynamic programming approach applicable if

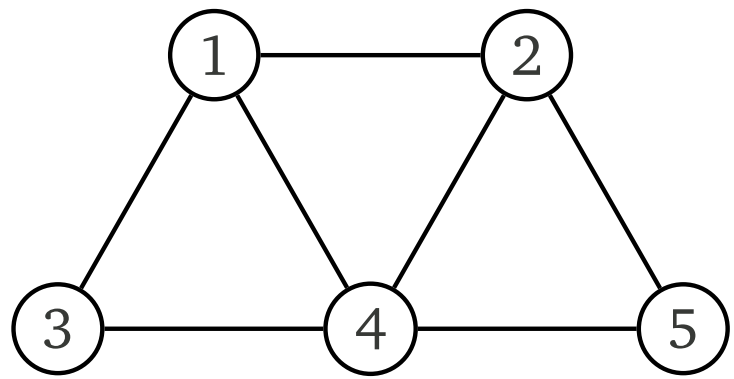
the problem has a certain "sub-structure" property that allows for a compositional solution.

Shortest-path problems have the property

if $x_1 – x_2 – \cdots – x_i – \cdots – x_n$ is a shortest path from x_1 to x_n then $x_1 – x_2 – \cdots – x_i$ is a shortest path from x_1 to x_i.

Longest-path problems don't have that property

1–3–4–2–5 is a longest path from 1 to 5, but 1–3–4–2 is not a longest path from 1 to 2 (since 1–3–4–5–2 is longer)



Summary

Today

- Dynamic Programming: Graphs
- Warshall and Floyd

Next

- Greedy algorithms